

02 - Dart 基礎語法

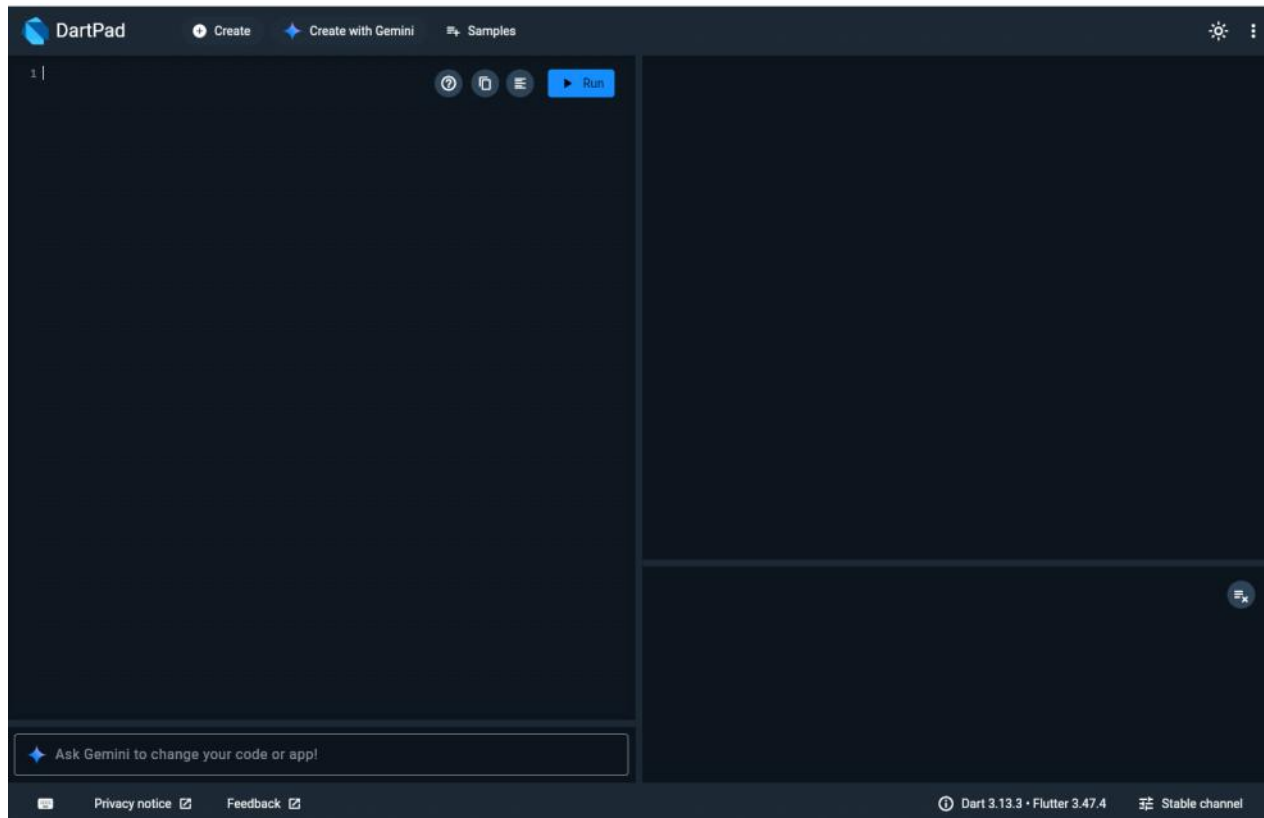
一、關於 Dart

Dart 是一種適用於全球資訊網的開放原始碼程式語言，由 Google 主導開發，於2011年10月公開，目標在於成為下一代結構化 Web 開發語言。Dart 也是一種物件導向語言，類似 JavaScript，但是它採用類別為基的程式設計。它只允許單一繼承，語法風格接近C語言。

二、Dart 基礎語法

(一) DartPad 線上工具

DartPad 是一個可以在任何瀏覽器中體驗 Dart 程式語言的線上開源工具。(<https://dartpad.dev/>)。



(二) 程式的起點：main() 與 print()

所有 Dart 程式都有入口：

1. main() 是程式執行起點。
2. print() 用來將文字或結果輸出在螢幕上。
3. // 後面的文字是註解，電腦會忽略它，寫給人類看的備忘錄。

```
void main() {  
  // 每一行指令結束，一定要加分號 ;  
  print('哈囉，世界！');  
}
```

(三) 變數 (資料的收納盒)

變數就像貼了標籤的盒子，用來放資料。

常見的 4 種基本型別：

1. int：整數（如 10, -5）
2. double：小數（如 3.14, 0.5）
3. String：文字（如 '小明', 'Dart'）
4. bool：布林值，只有兩種狀態：true（真/是）或 false（假/否）

```
void main() {  
    // 顯式寫出型別  
    String name = '小明';  
    int age = 18;  
    double height = 175.5;  
    bool isStudent = true;  
  
    // 用 var 讓電腦自己猜型別  
    var city = '台北'; // 電腦會自動猜這是 String  
  
    // 字串插值：用 $ 把變數放進句子裡  
    print('我是 $name，今年 $age 歲，身高 $height 公分。');  
    print('住在 $city，學生身分：$isStudent');  
  
    // 如果要做簡單計算，用 ${} 包起來  
    print('明年我就 ${age + 1} 歲了！');  
}
```

（四）常數：不能改的變數（final 與 const）

一般變數隨時可以換內容，但常數只要設了一次就不能再改。

常見的 4 種基本型別：

1. final：執行時才決定的值（例如：今天的日期、手機當前的電量）
2. const：在寫程式當下就完全確定的固定值（例如：圓周率 π 、一星期的天數 7）

```
// ignore_for_file: unused_local_variable  
void main() {  
    var score = 60;  
    score = 80; // 一般變數：可以改值  
  
    final DateTime today = DateTime.now(); // 程式跑起來時才知道今天幾號  
    // today = DateTime.now(); // 錯誤！final 不能被重新修改  
  
    const double pi = 3.14159; // 寫程式時就已經知道數值  
    // pi = 3.14; // 錯誤！const 也不能改  
}
```

（五）字串插值

```
void main() {
    String name = 'Tom';
    int age = 20;

    print('姓名: $name');
    print('年齡: $age');
}
```

練習一：

建立以下變數：

姓名、年齡、職業、身高、是否有程式設計經驗

最後輸出：

我是王小明，今年25歲

職業是網頁設計師

身高172.5公分

是否有程式經驗：true

(六) 運算子

1.算術運算子：+、-、*、/、%

```
void main() {
    int a = 10;
    int b = 3;

    print(a + b);
    print(a - b);
    print(a * b);
    print(a / b);
    print(a % b);
}
```

2.比較運算子：==、!=、>、<、>=、<=

```
void main() {
    int age = 20;

    print(age >= 18);
}
```

3.邏輯運算子：&&、||、!

```
void main() {
    int age = 20;
    bool hasTicket = true;

    print(age >= 18 && hasTicket);
}
```

(七) 條件判斷

1.if

```
void main() {
    int score = 80;

    if (score >= 60) {
        print('及格');
    }
}
```

2.if / else

```
void main() {
    int score = 50;

    if (score >= 60) {
        print('及格');
    } else {
        print('不及格');
    }
}
```

3.else if

```
void main() {
    int score = 85;

    if (score >= 90) {
        print('A');
    } else if (score >= 80) {
        print('B');
    } else if (score >= 70) {
        print('C');
    } else if (score >= 60) {
        print('D');
    } else {
        print('不及格');
    }
}
```

4.三元運算子

```
void main() {
    int age = 18;
    if (age >= 18) {
        print('成年');
    } else {
        print('未成年');
    }
}
```

簡化：

```
void main() {
    int age = 20;

    String result = age >= 18 ? '成年' : '未成年';

    print(result);
}
```

5.switch

```
void main() {
    String drink = '紅茶';

    switch (drink) {
        case '紅茶':
            print('你選擇紅茶');
            break;

        case '綠茶':
            print('你選擇綠茶');
            break;

        default:
            print('沒有這個選項');
    }
}
```

練習二：成績判斷

分數	等級
90~100	A
80~89	B
70~79	C
60~69	D
0~59	E

(八) 迴圈

1.for

```
void main() {
    for (int i = 1; i <= 5; i++) {
        print(i);
    }
}
```

2.while

```
void main() {
    int i = 1;

    while (i <= 5) {
        print(i);
        i++;
    }
}
```

3.break

```
void main() {
    for (int i = 1; i <= 10; i++) {

        if (i == 5) {
            break;
        }

        print(i);
    }
}
```

4.continue

```
void main() {
    for (int i = 1; i <= 5; i++) {

        if (i == 3) {
            continue;
        }

        print(i);
    }
}
```

練習三：1 到 100 加總

練習四：九九乘法表

(九) List、Set、Map 集合資料

1.List - 存放一組有順序的資料，可以進行新增、刪除、修改、查詢長度。

```
void main() {
    List<String> fruits = [
        'Apple',
        'Banana',
        'Orange',
    ];

    print(fruits);
}
```

2.Set - 不允許重複

```
void main() {
    Set<String> skills = {
        'HTML',
        'CSS',
        'Dart',
    };

    print(skills);
}
```

3.Map - 查字典 (Key 鍵 → Value 值)

```
void main() {
    Map<String, dynamic> user = {
        'name': 'Tom',
        'age': 25,
        'job': 'Engineer',
    };

    print(user['name']);
}
```

練習五：購物清單

```
void main() {

    List<String> cart = [
        '蘋果',
        '香蕉',
        '牛奶',
    ];

    cart.add('咖啡');

    for (String item in cart) {
        print(item);
    }

}
```

練習六：學生資料

```
void main() {

    Map<String, dynamic> student = {
        'name': '王小明',
        'score': 85,
        'passed': true,
    };

    print('姓名: ${student['name']}');
    print('分數: ${student['score']}');
}
```

(十) Function

1.無參數

```
void sayHello() {  
  print('Hello');  
}  
  
void main() {  
  sayHello();  
}
```

2.有參數

```
void sayHello(String name) {  
  print('Hello $name');  
}  
  
void main() {  
  sayHello('Tom');  
}
```

3.回傳資料

```
int add(int a, int b) {  
  return a + b;  
}  
  
void main() {  
  int result = add(10, 20);  
  print(result);  
}
```

4.箭頭函式

```
int add(int a, int b) {  
  return a + b;  
}  
  
int add(int a, int b) => a + b;
```

5.具名參數

```

void showUser({
  required String name,
  required int age,
}) {

  print('$name / $age');

}

void main() {
  showUser(
    name: 'Tom',
    age: 20,
  );
}

```

練習六：計算商品總價

```

double calculateTotal({
  required double price,
  required int quantity,
}) {
  return price * quantity;
}

void main() {

  double total = calculateTotal(
    price: 1200,
    quantity: 3,
  );

  print('總價: $total');

}

```

(十一) Class 、 Object 與物件導向

1.class

```

class Person {

  String name;
  int age;

  Person(this.name, this.age);

  void introduce() {
    print('我是 $name , 今年 $age 歲');
  }

}

```

2.extends

```
class Animal {  
  
    void eat() {  
        print('吃東西');  
    }  
  
}  
  
class Dog extends Animal {  
  
    void bark() {  
        print('汪汪');  
    }  
  
}
```

3.override

```
class Animal {  
  
    void sound() {  
        print('Animal sound');  
    }  
  
}  
  
class Dog extends Animal {  
  
    @override  
    void sound() {  
        print('汪汪');  
    }  
  
}
```

練習七：商品類別

```

class Product {
    String name;
    int price;

    Product(this.name, this.price);

    void showInfo() {
        print('$name : $price 元');
    }
}

void main() {
    Product keyboard = Product(
        '機械鍵盤',
        2500,
    );

    keyboard.showInfo();
}

```

(十二) Null Safety 與錯誤處理

1. 不允許 null

```
String name = null;
```

2. ? - 允許 null

```
String? name;
```

3. ?? - 如果是 null，使用預設值。

```
String? name;

void main() {
    print(name ?? 'Guest');
}

```

4. ! - 確定這個資料不是 null

```
String? name = 'Tom';

void main() {
    print(name!.length);
}

```

5. ?. - 安全呼叫

```
String? name;

void main() {
  print(name?.length);
}
```

5.try / catch

```
void main() {

  try {

    int result = 10 ~/ 0;

    print(result);

  } catch (e) {

    print('發生錯誤: $e');

  }

}
```

練習八：

```
void printUserName(String? name) {

  String displayName = name ?? '訪客';

  print('歡迎 $displayName');

}

void main() {

  printUserName(null);
  printUserName('Tom');

}
```

(十三) Future、async / await

1.Future - 現在還沒有資料，但未來會有。

```
Future<String> getData() async {

  await Future.delayed(
    Duration(seconds: 2),
  );

  return '資料讀取完成';

}
```

2.async - 這個函式裡面可能有非同步工作。

```
Future<void> loadData() async {  
}
```

3.await - 等待這個非同步工作完成。

```
String data = await getData();
```

4.結合

```
Future<String> getUser() async {  
  await Future.delayed(  
    Duration(seconds: 2),  
  );  
  return '王小明';  
}  
  
void main() async {  
  print('開始讀取');  
  String user = await getUser();  
  print('使用者: $user');  
  print('完成');  
}
```

練習九：購物車系統

```

void main() {
  List<Map<String, dynamic>> products = [
    {'name': '鍵盤', 'price': 1200, 'quantity': 2},
    {'name': '滑鼠', 'price': 800, 'quantity': 1},
    {'name': '螢幕', 'price': 5000, 'quantity': 1},
  ];

  for (var product in products) {
    print(
      '${product['name']}' '
      '${product['price']} 元 '
      'x ${product['quantity']}',
    );
  }

  int total = calculateTotal(products);

  print('-----');
  print('總金額: $total 元');
}

int calculateTotal(List<Map<String, dynamic>> products) {
  int total = 0;

  for (var product in products) {
    total += (product['price'] * product['quantity']) as int;
  }

  return total;
}

```

綜合實作：學生管理系統

資料結構：

```

List<Map<String, dynamic>> students = [
  {
    'name': '小明',
    'score': 85,
  },
  {
    'name': '小華',
    'score': 58,
  },
  {
    'name': '小美',
    'score': 92,
  },
];

```

完成項目：

1. 顯示所有學生姓名與分數
2. 判斷是否及格
3. 計算平均分數

4. 找出最高分
5. 使用 Function 分離功能
6. 至少使用一次 List
7. 至少使用一次 Map
8. 至少使用一次 if
9. 至少使用一次 for

輸出結果範例：

小明：85 分，及格
小華：58 分，不及格
小美：92 分，及格

平均分數：78.33

最高分：92